Explaining ECG Arrhythmia Classifiers with PermFIT Permutation Feature Importance

Group: H · **Case study:** Explainable Machine Learning – ECG arrhythmia detection

Name	Student ID	Name	Student ID
ZiMing Zhang	3202195Z	Zhihui HUANG	3107262H
Liangxiao HE	3267551H	Bhushan Laxman Mahajan	3209966M
Subhranshu Panda	3188758P		

1. Introduction

An electrocardiogram (ECG) records the heart's electrical activity. This is the most common and least expensive test to check for arrhythmia (irregular heart rate or rhythm). Each heartbeat has three parts. The P-wave is the depolarisation of the atria. The QRS complex is the depolarisation of the ventricles. The T-wave is the repolarisation of the ventricles. Interpreting ECGs manually is slow and error-prone: in a national sample of ~550,000 patients, approximately one-third of ECGs were misread (Wu et al., 2018). Machine learning (ML) can automatically classify beats. However, accurate models such as support vector machines (SVM) and random forests are black boxes: they do not explain why a beat is labelled as abnormal, which reduces the confidence of clinicians (Arrieta et al., 2020).

We use the MIT-BIH Arrhythmia Database (Moody and Mark, 2001), which contains 48 thirty-minute dual-channel recordings from 47 subjects, annotated beat by beat by cardiologists. We classify 8 types of beats, which are normal (N), left and right bundle branch block (L, R), premature ventricular contraction (V), atrial premature beat (A), fusion of ventricular and normal beat (F), fusion of paced and normal beat (f) and paced beat (/).

The study answers two questions:

- **Predictive:** How accurate are an SVM and a random forest in classifying the eight beat categories, and how much of this accuracy is retained when tested on previously unseen patients?
- **Exploratory:** What aspects of the heartbeat do the models rely on? Assessed with PermFIT permutation feature importance (Mi et al., 2021).

We hypothesise that the slices covering the QRS complex (slices 5–7 of 11) receive the highest importance.

2. Methods

2.1 Data and preprocessing

Both ECG channels were processed using the Python packages WFDB and BioSPPy to extract beats. Each beat is a 275-sample window (0.76 s at 360 Hz) centred on the detected R-peak. It is standardised to zero mean and unit variance (z-score), which removes amplitude differences between patients and leads, and labelled by the cardiologist annotation within ±10 samples of the peak. Other classes or no matching annotation beats were dropped. This leaves **149,768** beats. Each heartbeat appears once in each channel. The record number is stored with each beat so the data can be segregated by patient. The data are highly imbalanced, with normal beats making up 76% and fusion beats (F) only 0.5% (Figure 2).

2.2 Validation protocols and class balancing

- **Beat hold-out:** a random 75% and 25% split of all beats (112,326 training, 37,442 test). Beats of the same patient appear in both sets. The test score can be inflated by data leakage (Jones et al., 2020).
- **Patient hold-out:** all beats of records 104, 113, 119, 208 and 210 are kept for testing (14,482 beats), and the other 43 records are used for training (as in Jones et al., 2020). These five patients have no L, R or A beats.

In each training set, every class is resampled with scikit-learn's bootstrap *resample* (Pedregosa et al., 2011) to the mean size of the seven abnormal classes: 3,874 beats per class for beat hold-out (30,992 in total) and 4,423 per class for patient hold-out (35,384). Test sets keep the real class distribution. While integrating the code, we found and fixed a bug in the provided patient-split code. It built the list of training rows once per held-out patient and then joined the five copies, resulting in the training set containing every beat about five times, including the test patients' beats. This is why the handout reports 25,789 beats per class.

2.3 Classifiers

Support vector machine (Tasks 2–4). An SVM finds the boundary that separates classes with the largest possible margin. When using a radial basis function kernel, the SVM compares each beat to stored support beats using a Gaussian similarity measure. This allows the decision boundary to curve and adapt to patterns. Two key parameters control this process. The first is **C**, which sets the penalty for misclassifying training beats. A higher C value makes the model fit the training data closely even if it leads to overfitting. The second parameter is **gamma**, which controls the width of the similarity. In scikit-learn, the default setting for gamma is 'scale ', which calculates it as 1 divided by (275 * the variance of the training data). The classifier handles eight classes using a one-vs-one approach. For the beat hold-out, the model uses SVC with kernel='rbf' and the default C value of 1. For the patient hold-out, the same SVM model is used, but with a StandardScaler first. This scaler adjusts each time point to have unit variance, which helps normalize the data across patients. The SVM is trained with a maximum of 10,000 iterations.

Random forest (Task 5). This model uses a forest of 300 decision trees. Each tree is trained on a bootstrap sample of the data, and at every split it selects a subset of the 275 time points to consider. The final prediction is made by majority vote across all trees. This model serves as a contrast to the SVM. While the random forest makes decisions based on threshold questions about time points, the RBF SVM compares the overall shape of the beat using similarity across all time points. The number of time points considered at each split (max_features) and the minimum number of samples required in a leaf node were determined through grid search. This tuning was done using 3-fold stratified cross-validation on an 8,000-beat subset of the beat hold-out training data. The evaluation metric used was macro-F1.The test sets were never used during tuning. The SVM results are included in the table for reference. Section 4 provides discussion on the SVM.

Table 1. Grid search, 3-fold stratified cross-validation, macro-F1 (mean ± s.d.). The chosen setting is shaded.

Model	Parameters	CV macro-F1
SVM	{C: 1}	0.928 ± 0.002
SVM	{C: 10}	0.955 ± 0.003
SVM	{C: 100}	0.960 ± 0.001
Random Forest	{max_features: sqrt, min_samples_leaf: 1}	0.954 ± 0.001
Random Forest	{max_features: sqrt, min_samples_leaf: 3}	0.950 ± 0.004
Random Forest	{max_features: 0.1, min_samples_leaf: 1}	0.954 ± 0.001
Random Forest	{max_features: 0.1, min_samples_leaf: 3}	0.950 ± 0.003

2.4 Evaluation metrics

We report accuracy and the per-class precision, recall and F1-score. Their macro average gives every class weight even though there is an imbalance. For the patient hold-out the macro

average uses only the five classes present in the test set (N, V, F, f, /) because recall is undefined for absent classes. Row-normalised confusion matrices show where the beats of each class are assigned.

2.5 Explainability: PermFIT

Permutation feature importance measures how a fitted model's error increases when one input feature is destroyed. If the model relies on that feature, breaking its link with the label harms the predictions. It is model-agnostic and gives a global explanation for the whole dataset. PermFIT (Mi et al., 2021) adds K-fold cross-fitting. Every beat is scored by a model that did not see it during training. Here, the features are 11 slices of 25 samples. Slices 1–4 cover the PR interval; 5–7 the QRS complex (the R-peak lies in slice 6); 8–11 cover the ST segment and T-wave.

1. Split the training set into K = 5 folds, stratified on the beat class.
2. For each fold k, fit a new model on the other four folds.
3. For every beat i in fold k and every slice j, perturb slice j and compute the importance
$$\hat{M}_{ij} = \Sigma_c \left(Y_{ic} - \hat{p}_c(X_i^{(j)}) \right)^2 - \Sigma_c \left(Y_{ic} - \hat{p}_c(X_i) \right)^2$$
where Y_i is the one-hot label of beat i, $\hat{p}(\cdot)$ the predicted class probabilities, and $X_i^{(j)}$ the beat with slice j perturbed. A positive value means the prediction got worse.
4. The importance of slice j is the mean over all beats, $\bar{M}_j$; the spread across the five folds is shown as error bars. As instructed, variance and p-values are not estimated.

Perturbation. As the case study asks, the main method **shuffles the 25 samples inside the slice** at random, doing this separately for each beat. This action breaks the shape of the slice while leaving its mean and variance unchanged. To test sensitivity, we also set the slice to zero, replace it with its mean (a flat line) and swap it with the same slice from another random beat (classic permutation importance). The three additional methods reuse the fitted fold models. **Probabilities**. An SVM does not produce probabilities, so its scores were calibrated using Platt scaling (Platt, 1999; scikit-learn's CalibratedClassifierCV with ensemble=False), which preserves the SVM's decision boundary. For the forest, the probability is calculated as the fraction of tree votes. We also report the drop in accuracy expressed in percentage points (pp) that results from perturbing each slice, which is easier to interpret. All random steps use seed 42.

3. Results

3.1 Experimental setup

Table 2. Experiments and their rationale.

Experiment	Training data	Test data	Rationale
E1 – SVM, beat hold-out (Task 2)	30,992 beats, 48 records	37,442 beats	Standard protocol; an optimistic upper bound because of leakage
E2 – SVM, patient hold-out (Task 3)	35,384 beats, 43 records	14,482 beats, 5 unseen records	Realistic test on new patients; E1 − E2 shows the leakage effect
E3 – PermFIT on the E1 and E2 SVMs (Task 4)	5 stratified folds of each training set (≈24.8k / 6.2k and 28.3k / 7.1k beats per fold)	Held-out fold	Which parts of the beat does the SVM use?
E4 – Random forest + PermFIT, both protocols (Task 5)	As E1–E3	As E1–E3	Do performance and explanation depend on the classifier?
E5 – Four perturbation methods	Same fold models as E3–E4	Held-out fold	Does the explanation depend on how a slice is destroyed?

3.2 The data

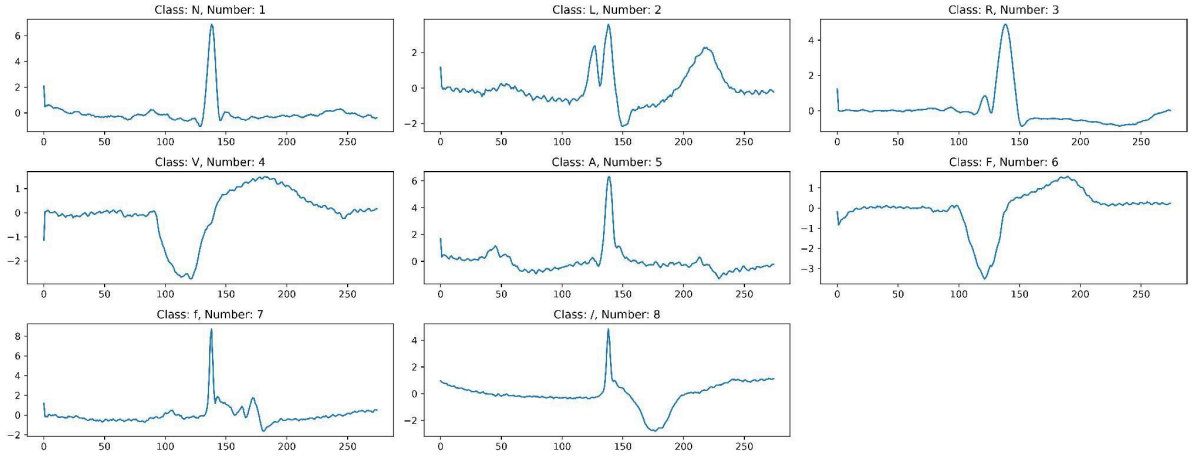


Figure 1. One preprocessed beat of each class (Task 1). Beats are 275 samples long and centred on the R-peak (sample 137). The classes differ most in the width and shape of the QRS complex.

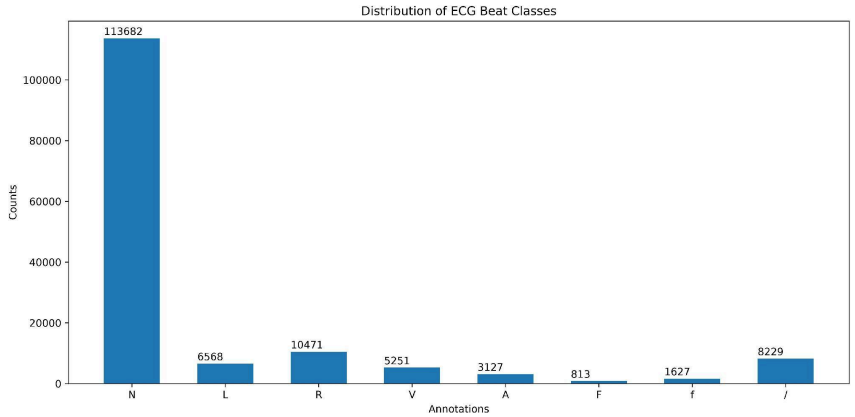


Figure 2. Beats per class in the preprocessed data (both channels). Normal beats dominate; F and f are rare.

3.3 Classification (Tasks 2, 3 and 5)

Table 3. Test-set performance. Macro scores average the classes present in the test set. The last two columns show the recall of the two fusion classes.

Model	Protocol	Accuracy	Macro precision	Macro recall	Macro F1	Recall F	Recall f
SVM (Task 2)	Beat hold-out	0.938	0.763	0.953	0.815	0.932	0.950
SVM (Task 3)	Patient hold-out	0.866	0.843	0.692	0.694	0.452	0.125
Random forest	Beat hold-out	0.968	0.850	0.954	0.893	0.890	0.930
Random forest	Patient hold-out	0.836	0.791	0.581	0.529	0.003	0.004

On the **beat hold-out** both models achieve over 93% accuracy. Every class has a recall of at least 0.89. Precision on rare classes is low. The SVM mislabels 818 test beats as F. That means the precision for F is only 0.17 (F1 = 0.29). The training data are balanced. The test data are 76% normal. So even a small false-positive rate on N has a big impact on the rare class.

On the **patient hold-out**, accuracy drops by 7.2 percentage points for the SVM (93.8% → 86.6%) and by 13.2 percentage points for the forest (96.8% → 83.6%). The macro-F1 score falls from 0.815 to 0.694 for the SVM and from 0.893 to 0.529 for the forest. Normal beats, V beats and paced beats are still recognized well (recall ≥ 0.91, ≥ 0.97 and ≥ 0.97). Fusion classes break down.

Figure 3 shows that unseen F beats are often assigned to N and V, the two beat types they come from. Unseen f beats end up spread across all classes. **The random forest performs better when patients are shared between training and test. It performs worse when the test data come from new patients.**

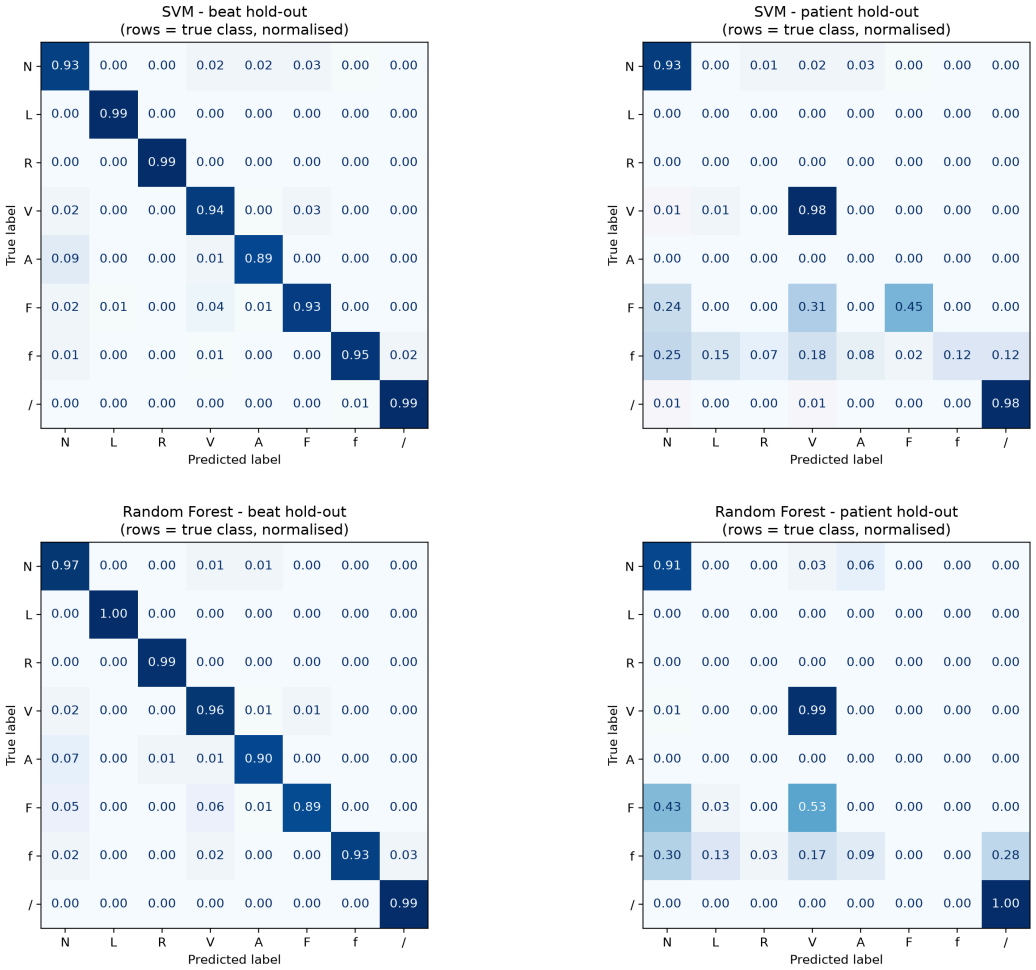


Figure 3. Row-normalised confusion matrices (rows = true class). Left: beat hold-out; right: patient hold-out (L, R and A are absent from that test set). Top: SVM; bottom: random forest.

3.4 PermFIT explanation of the SVM (Task 4)

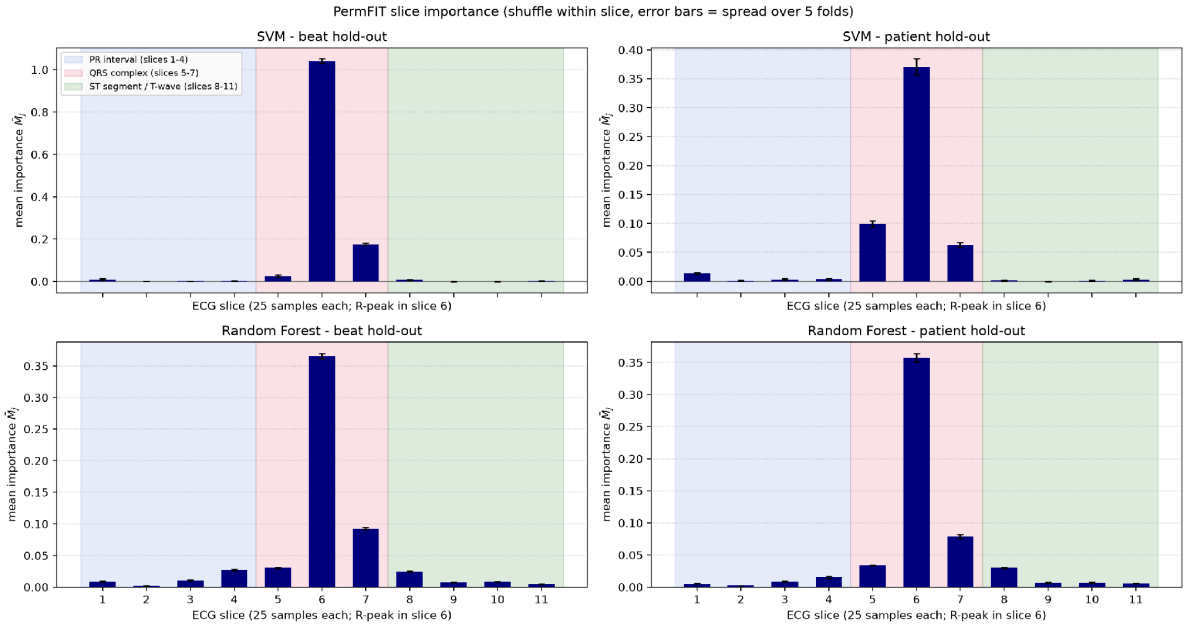


Figure 4. PermFIT slice importance (shuffle within slice) for both models and both protocols. Shading marks the PR interval (blue), the QRS complex (red) and the ST segment / T-wave (green). Error bars = standard deviation over the 5 folds.

Table 4. PermFIT mean importance $\bar{M}_j$ per slice (shuffle within slice), the share of positive importance in the QRS slices, and the drop in accuracy when slice 6 is shuffled.

Slice (region)	SVM, beat	SVM, patient	RF, beat	RF, patient
1 (PR)	0.009	0.014	0.009	0.005
2 (PR)	0.000	0.001	0.002	0.002
3 (PR)	0.001	0.004	0.011	0.008
4 (PR)	0.003	0.004	0.027	0.015
5 (QRS)	0.025	0.099	0.030	0.034
6 (QRS)	1.040	0.370	0.365	0.357
7 (QRS)	0.176	0.063	0.093	0.079
8 (ST/T)	0.007	0.002	0.025	0.030
9 (ST/T)	0.000	0.000	0.008	0.007
10 (ST/T)	-0.001	0.001	0.009	0.007
11 (ST/T)	0.002	0.003	0.005	0.006
QRS share (slices 5–7)	98.2%	94.9%	83.7%	85.3%
Accuracy drop, slice 6	57.8 pp	20.4 pp	24.8 pp	20.5 pp

For both SVMs, slice 6, which contains the R-peak, dominates. Shuffling it raises the squared error by 1.040 (beat hold-out model) and 0.370 (patient hold-out model), and lowers accuracy by 57.8 and 20.4 percentage points. The QRS slices 5–7 together hold 98.2% and 94.9% of the total importance; the PR and ST/T slices contribute very little. The small error bars show that the result is stable across folds. Two details stand out. First, the patient hold-out SVM gives more weight to slice 5, the onset of the QRS complex (0.099 against 0.025). Second, **slice 1 is the fourth most important slice for both SVMs**, even though it contains no physiological event. It does contain the edge artefact described in Section 4.

3.5 SVM versus random forest (Task 5)

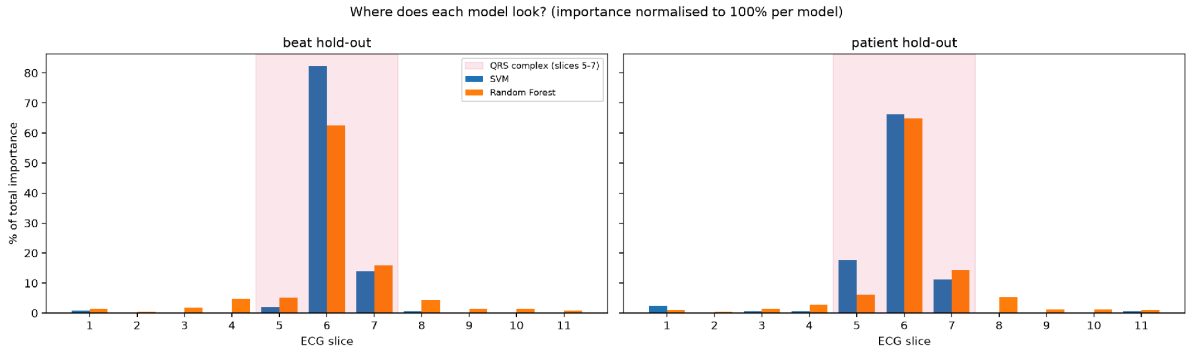


Figure 5. Share of total importance per slice for the SVM and the random forest, for each protocol.

The random forest also ranks slices 6, 7 and 5 as highest under both protocols. Its importance is less concentrated: 83.7% and 85.3% are in the QRS slices. Instead, it also uses slices 4 and 8, the end of the PR segment and the start of the ST segment just near the QRS complex. Their importance is 0.027 and 0.025 for the random forest compared to 0.003 and 0.007 for the SVM (beat hold-out). The pattern is very similar for both protocols (Pearson r = 0.999, between the beat and patient profiles). Slice 1 does not matter much for the random forest.

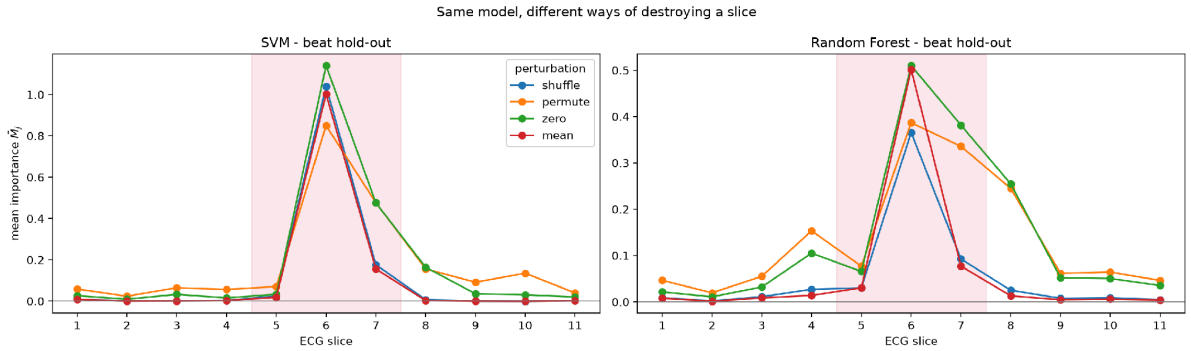


Figure 6. Mean importance per slice under four perturbation methods (beat hold-out models).

The choice of perturbation changes the picture (Figure 6). Shuffling and mean replacement give almost the same profile, because both keep the slice's average level and only destroy its shape. Setting the slice to zero or swapping it with another beat also changes the level, so the ST segment becomes important too. For example, zeroing slice 8 lowers the random forest's accuracy by 19.6 points, and the QRS share drops to 83% (SVM) and 63% (random forest). Across all 16 combinations of model, protocol, and perturbation, slice 6 remains the most important slice.

4. Discussion

Accuracy depends strongly on the validation protocol. The beat hold-out results (94–97% accuracy) are higher than the patient hold-out results (84–87% accuracy), showing that evaluating the models only on randomly selected beats can give an overly optimistic view of their performance. The largest drop occurs in the fusion classes, F and f. These classes are rare and concentrated in a small number of patients: 92% of F beats come from records 208 and 213, while 70% of f beats come from record 104. Since some of these records are held out for testing, the models have limited examples from other patients to learn from. This makes it difficult to recognise these classes reliably in unseen patients. These findings support the need for patient-level validation and are consistent with Jones et al. (2020).

Both models rely mainly on the QRS complex, supporting our hypothesis. Between 84% and 98% of the PermFIT importance lies in slices 5–7, with the R-peak slice being the most important across all experiments. This is consistent with the role of the QRS complex in

identifying arrhythmias, as conditions such as bundle branch blocks and ventricular beats can change its shape and width. The results therefore suggest that the models rely on an ECG region with clinical significance.

The explanation also revealed a preprocessing artefact. The preprocessing cuts a 274-sample window around each R-peak and pads it with one zero to reach 275 samples, so the first sample of every beat is padding. After standardisation, this zero becomes an artificial spike at sample 0, inside slice 1. The SVM ranks slice 1 fourth in importance even though it contains no physiological event, which suggests it uses this spike. This shows how explainability can reveal problems in the data pipeline, not just describe model behaviour.

The classifiers use the ECG signal differently. The random forest distributes more importance towards the edges of the QRS complex (slices 4 and 8), while the RBF SVM concentrates more strongly around the R-peak. The random forest performs better on the beat hold-out split, but the SVM performs better on the patient hold-out split. This suggests that model performance depends on the validation setting. However, the two SVM models also use different preprocessing, so their explanation profiles cannot be compared as a direct measure of the effect of the validation protocol.

Limitations of the study. There are also some limitations related to the dataset and validation approach. The study uses 48 ECG recordings from 47 subjects, so the findings may not represent the full range of patients and arrhythmias. Furthermore, the patient hold-out test set contains only five of the eight classes, meaning that the models' ability to recognise L, R and A in unseen patients could not be evaluated. The beat-level split also treats observations from the two ECG channels separately rather than grouping them by their original heartbeat. As a result, closely related observations may appear in both the training and testing sets, creating a potential source of data leakage.

PermFIT also has some limitations. Since upsampling uses sampling with replacement, duplicate copies of the same beat may appear in different cross-validation folds, potentially making the results optimistic. Grouping the PermFIT folds by patient and original beat identity would provide a more reliable assessment.

Future work. Further work could include patient-grouped cross-validation, class-specific PermFIT to identify which ECG regions distinguish particular arrhythmias, and local explanations using LIME or SHAP. A one-dimensional CNN could also be explored. Although the grid search suggests that a larger SVM C value may improve performance (C = 100 achieved a CV macro-F1 of 0.960 compared with 0.928 for C = 1), the default value was retained so that the explained models matched those evaluated in Tasks 2 and 3.

Conclusion. Both the SVM and random forest achieve high accuracy when patients are shared between training and testing, but their performance decreases on unseen patients, particularly for the rare fusion classes. PermFIT identifies the QRS complex as the most important region for both models and also reveals a potential preprocessing artefact affecting the SVM. Overall, the results demonstrate the value of combining model performance evaluation with explainability when developing ECG classifiers.

Acknowledgements

This work used the generative AI assistant Claude (Opus 5.5 Anthropic, used through Claude Code) and ChatGPT (GPT 4o-mini OpenAI, used through the web). We used them to explain the lecture material and code, to help write and debug the code for Tasks 4 and 5 (PermFIT and the Random Forest comparison) and combine the group's scripts, and to help draft and edit parts of this report. The group checked, ran and edited all AI output, and all results come from code we ran ourselves.

References

Arrieta, A.B. et al. (2020) 'Explainable Artificial Intelligence (XAI): concepts, taxonomies, opportunities and challenges toward responsible AI', *Information Fusion*, 58, pp. 82–115.

Jones, Y., Deligianni, F. and Dalton, J. (2020) 'Improving ECG classification interpretability using saliency maps', in *2020 IEEE 20th International Conference on Bioinformatics and Bioengineering (BIBE)*, pp. 675–682.

Mi, X., Zou, B., Zou, F. and Hu, J. (2021) 'Permutation-based identification of important biomarkers for complex diseases via machine learning models', *Nature Communications*, 12, 3008.

Moody, G.B. and Mark, R.G. (2001) 'The impact of the MIT-BIH Arrhythmia Database', *IEEE Engineering in Medicine and Biology Magazine*, 20(3), pp. 45–50.

Pedregosa, F. et al. (2011) 'Scikit-learn: machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.

Platt, J. (1999) 'Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods', in *Advances in Large Margin Classifiers*. MIT Press, pp. 61–74.

Wu, J. et al. (2018) 'Impact of initial hospital diagnosis on mortality for acute myocardial infarction: a national cohort study', *European Heart Journal: Acute Cardiovascular Care*, 7(2), pp. 139–148.

Appendix A – Code and reproducibility

All code is in the submitted folder; README.md lists every file. walkthrough.ipynb runs Tasks 1-5 in order with short explanations and shows every result inline (outputs saved). Running *bash run_all.sh* rebuilds every figure and table from the raw MIT-BIH files with fixed random seeds. The main pieces are:

- *00_preprocess.ipynb*, *01_split_resample.ipynb*: the course notebooks with fixes (data paths, NumPy 2 compatibility, the patient-split bug).
- *task1.py*, *task2_svm.py*, *task3.py*: Load and inspect the data, plot a few examples from different arrhythmia types, Classification of ECG beats based on hold-out splitting method and leave-out patients hold-out validation protocol.
- *task4_permfit.py*, *task5_compare_classifiers.py*, *tune_hyperparameters.py*: PermFIT, the random forest comparison and the grid search; *ecg_utils.py* holds the shared PermFIT and evaluation code.

Appendix B – Contributions

Member	Contribution
Ziming Zhang	Task 1 – data inspection (task1.py)
Liangxiao HE	Task 2 – beat hold-out SVM (task2_svm.py)
Zhihui HUANG	Task 3 – patient hold-out SVM (task3.py)
Subhranshu and Bhushan	Task 4 – PermFIT, Task 5 – random forest comparison and code integration
Everyone	Report writing and review